SOAR Lite - Code Explanation Guide

File-by-file explanation of the project code

1. streamlit_app.py

This is the front-end dashboard. It uses Streamlit to create a SOC-style control room. It stores an Orchestrator instance in st.session_state so each browser session keeps its own in-memory workflow. The file defines the UI tabs: Start here, Threat checker, New simulation, Approval queue, and Audit trail. Important functions include create_demo_scenario(), create_account_compromise_scenario(), show_impact(), and show_inspection(). These functions display the simulated actions, risk impact, and suspicious-content findings.

2. main.py

This is the root server launcher. It imports the FastAPI app from soar_lite.main and starts it using uvicorn. The file is tiny but important because it is the entry point when running the backend from the project root. In practical terms, it tells Python: "run the app defined in soar_lite.main".

3. soar_lite/main.py

This is the real FastAPI backend. It creates the app object and defines each REST endpoint. The routes include /health, /playbooks, /actions, /actions/{id}, /actions/{id}/decision, /actions/{id}/rollback-simulation, and /audit. Each endpoint simply delegates to the orchestrator service, which keeps all workflow state in memory. This file is the communication layer between the UI and the internal logic.

4. soar_lite/models.py

This file holds the data schema of the project. It defines enums such as PlaybookName, ActionStatus, and RiskLevel, along with the Pydantic request and response models. The most important model is ContainmentAction: it stores the action_id, target, playbook, rationale, requested_by, status, impact_simulation, created_at, and decision metadata. This file answers the question: ?What data does the system store??

5. soar_lite/playbooks.py

This file contains simulate_impact(playbook, target). It returns a mock impact assessment for each supported playbook: host_isolation, session_revocation, and domain_block. It creates summary text, affected resources, expected effects, potential side effects, and a risk level. It is intentionally synthetic and safe: it never performs a real isolation or block action.

6. soar_lite/service.py

This is the core workflow engine. The Orchestrator class stores actions in a dictionary and an AuditLog object. The main methods are create_action(), decide(), simulate_rollback(), get_action(), and list_actions().

create_action() creates a new action in pending_human_approval and logs trigger_received, dry_run_completed, and approval_requested. decide() either approves or rejects the action. simulate_rollback() records a rollback-only in-memory step for approved actions. This is the heart of the SOAR logic.

7. soar_lite/audit.py

This is the in-memory structured event log. The AuditLog class stores AuditEvent schemas with event_id, action_id, event_type, timestamp, actor, and details. It supports record(), list_events(), and clear(). Every workflow step is written here, making the project auditable even though no real external systems are changed.

8. soar_lite/inspector.py

This file implements the suspicious-content detector. It scans text, URLs, PDF content, QR/barcodes, and phone numbers locally using regex, URL parsing, OCR-like text extraction, and QR decoding. Functions include extract_urls(), inspect_url(), inspect_text(), inspect_email(), inspect_pdf(), inspect_qr_image(), and inspect_phone_numbers(). The system never opens external links or queries a real threat feed; it produces local heuristics only.

9. tests/test_api.py

This file verifies the approval-gated workflow using FastAPI TestClient. The tests ensure that every action starts in pending_human_approval, that approval transitions the status to executed_mock, that rejection prevents further execution, and that rollback is invalid unless execution already occurred. It acts as proof that the workflow logic matches the intended behavior.

10. requirements.txt

This lists the package dependencies required to run the project: FastAPI, uvicorn, pydantic, pytest, Streamlit, pypdf, Pillow, and zxing-cpp. These libraries support the backend API, UI, PDF reading, QR decoding, and testing environment.

Simple workflow summary

1. A user creates an action request. 2. The backend calculates a synthetic impact. 3. The action waits for human approval. 4. The reviewer approves or rejects it. 5. The system logs every event. 6. Optional rollback is only simulated in memory. No real systems are changed.

This guide is designed to help explain the code clearly during a presentation or viva.